

// Day 28 of 30

TUTORIAL

SOLID Principles With Real Java Code

→ Swipe through all 6 slides



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

1/8

S — Single Responsibility

One class = one reason to change

BAD:

```
class UserService {  
    void saveUser() { ... }  
    void sendEmail() { ... }  
    void generateReport() { ... }  
}
```

GOOD:

```
class UserService { void saveUser() {} }  
class EmailService { void sendEmail() {} }  
class ReportService { void generate() {} }
```

💡 If 'and' fits in class description, split it



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

O — Open/Closed

Open for extension, closed for modification

BAD:

```
if (type == "CREDIT") { ... }  
else if (type == "DEBIT") { ... }  
else if (type == "UPI") { ... }  
// Add new type? Modify this file.
```

GOOD:

```
interface Payment { void pay(); }  
class CreditPay implements Payment { ... }  
class UPIPay implements Payment { ... }  
// New type? Add new class. No changes.
```

💡 New feature = new class, not modified old code



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

L — Liskov Substitution

Subtypes must behave like parent

BAD:

```
class Bird { void fly() {} }  
class Penguin extends Bird {  
    void fly() {  
        throw new Exception("Can't fly!");  
    }  
}  
// Penguin breaks Bird's contract
```

GOOD:

```
interface Flyable { void fly(); }  
class Eagle implements Flyable { ... }  
// Penguin doesn't implement Flyable
```

💡 If subclass breaks parent behavior, wrong hierarchy



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

I — Interface Segregation

Don't force classes to implement unused methods

BAD:

```
interface Worker {  
    void code();  
    void test();  
    void deploy();  
    void manage();  
}  
// Junior dev forced to implement manage()
```

GOOD:

```
interface Coder { void code(); }  
interface Tester { void test(); }  
interface Manager { void manage(); }  
// Implement only what you need
```

💡 Many small interfaces > one fat interface



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)

D — Dependency Inversion

Depend on abstractions, not implementations

BAD:

```
class OrderService {  
    private MySQLRepo repo = new MySQLRepo();  
    // Tightly coupled to MySQL  
}
```

GOOD:

```
class OrderService {  
    private final OrderRepo repo;  
    OrderService(OrderRepo repo) {  
        this.repo = repo;  
    }  
}  
// Can inject MySQL, Postgres, or Mock
```

💡 Spring's @Autowired is Dependency Inversion in action



Shubham Bhati

linkedin.com/in/bhatishubham

SOLID in Spring Boot

You're already using it

S → @Service, @Controller, @Repository

Each class = one responsibility

O → Strategy pattern with interfaces

L → JPA repositories all substitutable

I → Spring's many small interfaces

D → Constructor injection everywhere

Spring Boot was BUILT on SOLID.

💡 SOLID isn't theory. You use it every day in Spring.



done = true

Follow for more!

Like · Comment · Repost

Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)



Shubham Bhati

[linkedin.com/in/bhatishubham](https://www.linkedin.com/in/bhatishubham)